

**SRI CHANDRASEKHARENDRA SARASWATHI VISWA
MAHAVIDYALAYA**

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



Name : Reshmitha

Reg. No : 11249a339

Class : II B.E. (CSE)

Course Code: BCSF184P90

Course Name: Data science toolkit lab

**SRI CHANDRASEKHARENDRA SARASWATHI VISWA
MAHAVIDYALAYA**

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



BONA FIDE CERTIFICATE

This is to certify that this is the bonafide record of work done by
Mr/Ms. Reshmitha

with Reg. No 11249a339 of H-B.E.(CSE) during the
academic year 2025 – 2026.

Station: Ennathur

Date:

Staff-in-charge

Head of the Department

Submitted for the Practical examination held on _____

Examiner-1

Examiner-2

SL.NO	Program Name	Date	Page no	Signature
1.1.	Numpy operations.	8/02		
1.2.	Pandas.	8/02		
1.3	matplot.	8/02		
1.4.	Scikit learn.	8/02		
02	. Data exploration	14/02		
	And preprocessing			
03.	Linear regression.	14/02		
04	naive bayes.	23/02		
05.	Implement regularised			
	Logistic regression	02/03		
06	build models using dif			
	Ensemble techniques	09/3		
07.	Implement decision	09/03		
	Tree algo for			
	classification and			
	regression			

Reshmitha 339

Ask Gemini

colab.research.google.com/drive/1TcdONfokdCWxnjebkHfiDagqA3GWRvs

Reshmitha 339

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

Python 3

11

```
import numpy as np

# 1. ARRAY
arr = np.array([1, 2, 3, 4, 5])
print("Array:", arr)

# 2. Linspace (start, stop, number of values)
lin = np.linspace(0, 10, 5)
print("Linspace:", lin)

# 3. WHERE (condition check)
cond = np.where(arr > 3)
print("Where (arr > 3):", cond)

# 4. EYE (Identity matrix with offset)
eye_mat = np.eye(3)
print("Eye Matrix:\n", eye_mat)

# 5. ARANGE (start, stop, step)
ar = np.arange(0, 10, 2)
print("Arange:", ar)

# 6. IDENTITY (square identity matrix)
iden = np.identity(3)
print("Identity Matrix:\n", iden)

# 7. MIN and MAX
print("Min:", np.min(arr))
print("Max:", np.max(arr))

# 8. SHAPE
mat = np.array([[1, 2, 3], [4, 5, 6]])
print("Shape of matrix:", mat.shape)

# 9. RESHAPE
reshaped = np.reshape(arr, (5, 1))
print("Reshaped Array:\n", reshaped)

# 10. SIZE (total elements)
print("Size of array:", arr.size)

# 11. SUM (sum of elements)
sum_result = np.sum(arr)
print("Sum:", sum_result)

# 12. ARRAY OPERATIONS
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print("Array Addition:", a + b)
print("Array Subtraction:", a - b)
print("Array Multiplication:", a * b)
print("Array Division:", a / b)
```

Array: [1 2 3 4 5]

Linspace: [0. 2.5 5. 7.5 10.]

Where (arr > 3): (array([3, 4]),)

Eye Matrix:
[[1. 0. 0.]
[0. 1. 0.]
[0. 0. 1.]

Arange: [0 2 4 6 8]

Variables

Terminal

31°C
Sunny

Search

ENG
IN

10:00
11-04-2026

Reshmitha 339 - Colab

colab.research.google.com/drive/1TckdONfoklcWxnjebkHfiDagqA3GWRvs

Reshmitha 339

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all

```
# 10. SIZE (total elements)
print("\nSize of array:", arr.size)

# 11. EINSUM (sum of elements)
einsum_result = np.einsum('i->', arr)
print("\nEinsum (sum):", einsum_result)

# 12. ARRAY OPERATIONS
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print("\nArray Addition:", a + b)
print("Array Subtraction:", a - b)
print("Array Multiplication:", a * b)
print("Array Division:", a / b)
```

Array: [1 2 3 4 5]

Linspace: [0. 2.5 5. 7.5 10.]

Where (arr > 3): (array([3, 4]),)

Eye Matrix:

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

Arange: [0 2 4 6 8]

Identity Matrix:

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

Min: 1
Max: 5

Shape of matrix: (2, 3)

Reshaped Array:

```
[[1]
 [2]
 [3]
 [4]
 [5]]
```

Size of array: 5

Einsum (sum): 15

Array Addition: [5 7 9]
Array Subtraction: [-3 -3 -3]
Array Multiplication: [4 10 18]
Array Division: [0.25 0.4 0.5]

Variables Terminal

1 31°C Sunny

Search

1.2nd experiment PANDAS

Untitled10.ipynb - Colab

colab.research.google.com/drive/1TcdONfokdCWxnjebkHfiDagqA3GWRvs

Reshmitha 339

```
import pandas as pd
data = {'name': ['Alice', 'Bob', 'Charlie'],
        'age': [25, 30, 35],
        'city': ['US', 'GOA', 'PARIS']}
df = pd.DataFrame(data)
print(df.head())
print(df.info())
print(df.describe())
```

```
name  age  city
0  Alice   25   US
1    Bob   30  GOA
2  Charlie  35  PARIS
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   column  non-null count  dtype
---  ---
 0   name    3 non-null  object
 1   age     3 non-null  int64
 2   city    3 non-null  object
dtypes: int64(1), object(2)
memory usage: 284.0+ bytes
```

	Age	City
count	3.0	
mean	30.0	
std	5.0	
min	25.0	
25%	27.5	
50%	30.0	
75%	32.5	
max	35.0	

31°C Sunny

Search

ENG IN

09:54 11-04-2026

1.3 MATPLOTT

```
[2] ✓
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

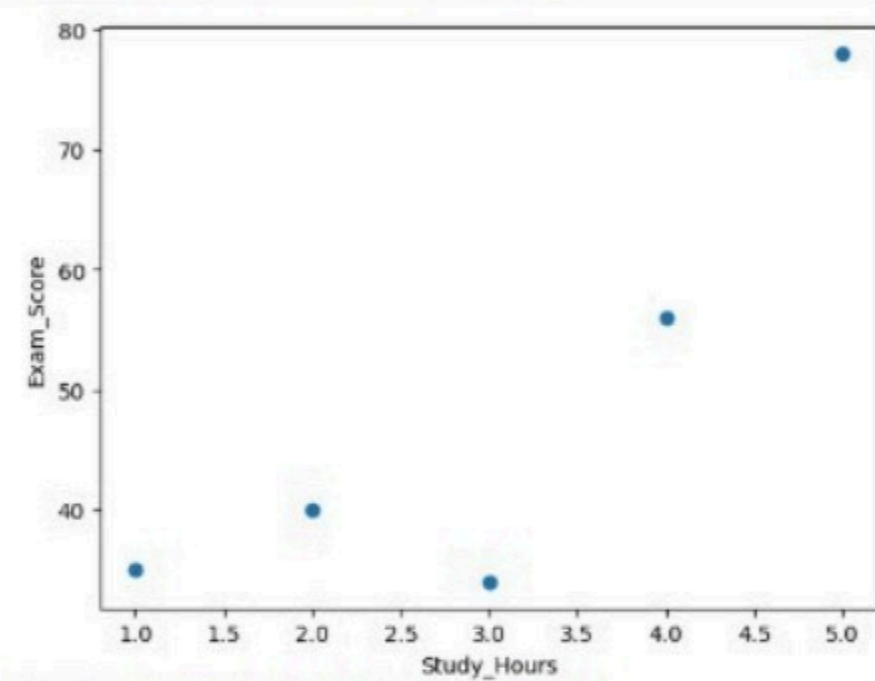
data = {'Study_Hours': [1,2,3,4,5],
        'Exam_Score': [35,40,34,56,78]}

df = pd.DataFrame(data)

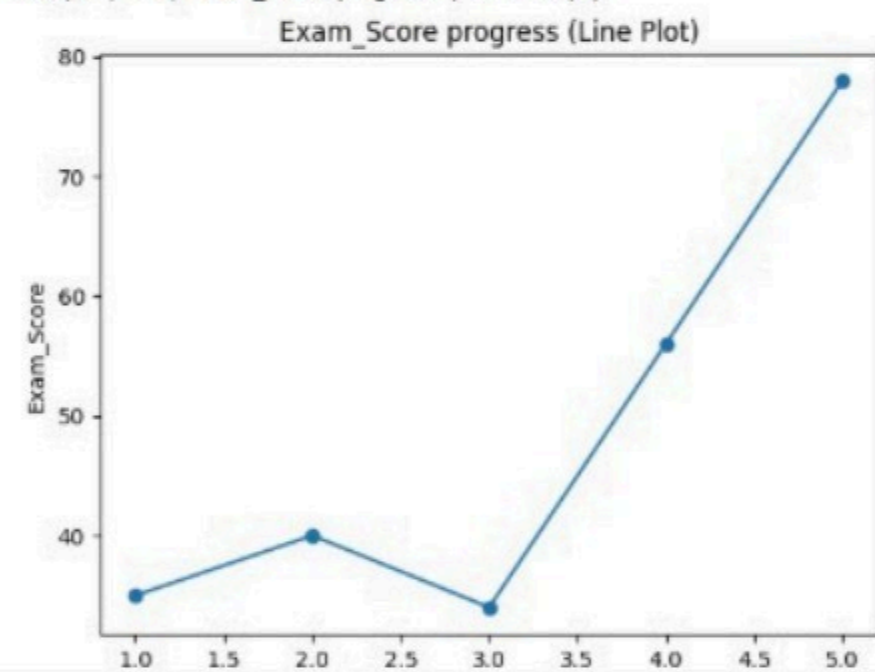
plt.scatter(df['Study_Hours'], df['Exam_Score'])

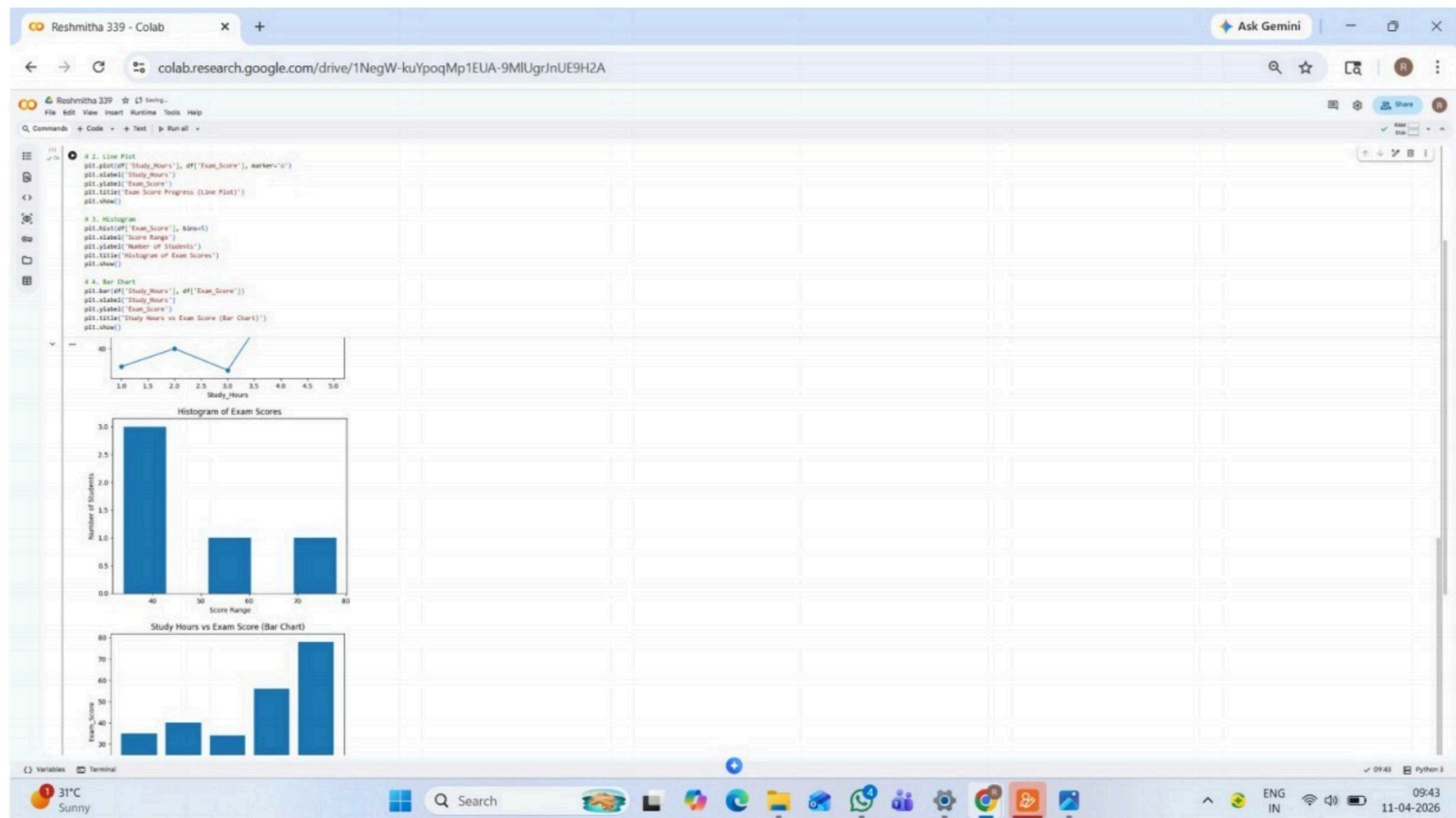
plt.xlabel('Study_Hours')
plt.ylabel('Exam_Score')

plt.show()
plt.plot(df['Study_Hours'], df['Exam_Score'], marker = 'o')
plt.xlabel('Study_Hours')
plt.ylabel('Exam_Score')
plt.title('Exam_Score progress (Line Plot)')
```

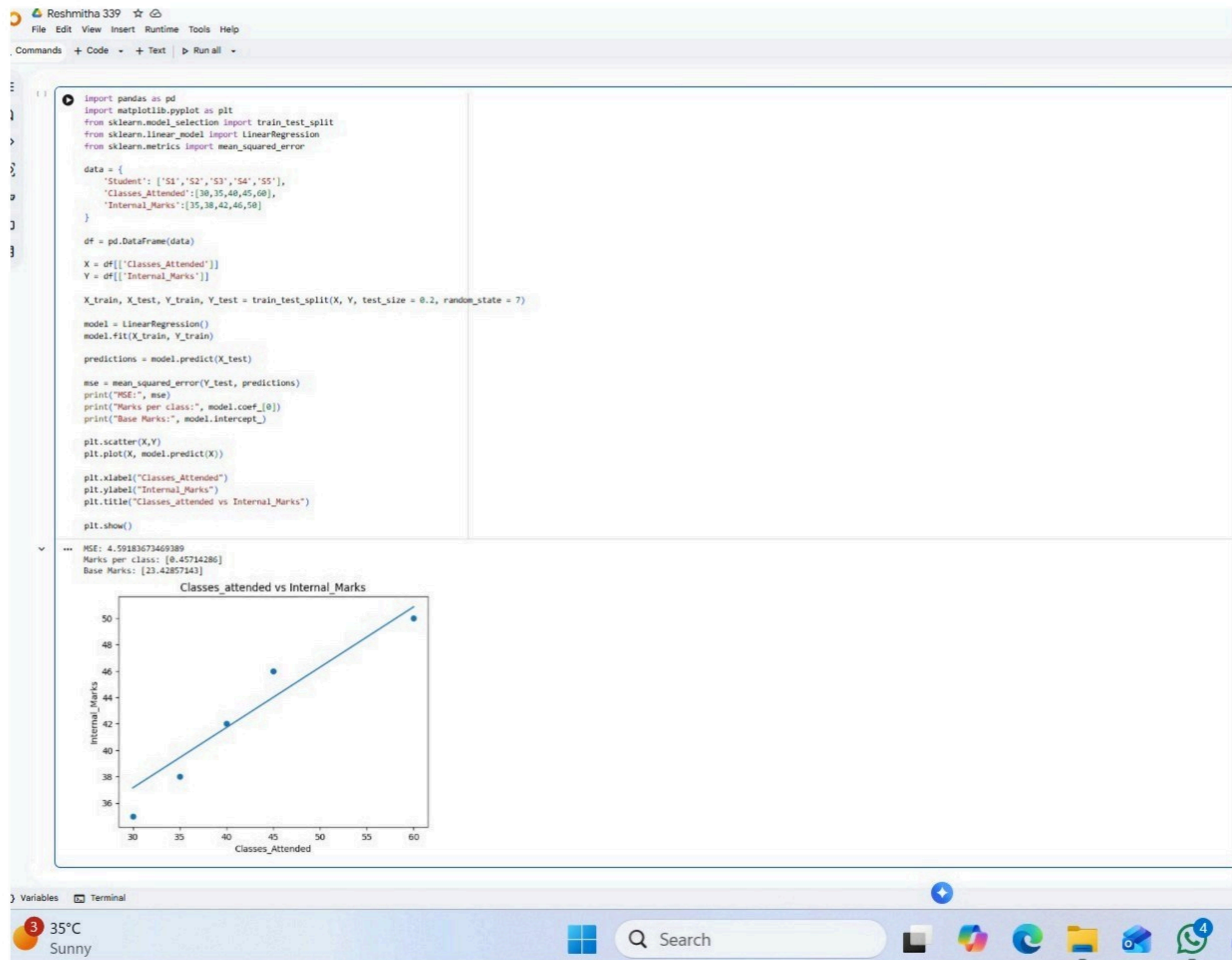


Text(0.5, 1.0, 'Exam_Score progress (Line Plot)')





1.4 SCIKIT LEARN



2 ND EXPERIMENT DATA EXPLORATION AND PREPROCESSING

colab.research.google.com/drive/19f9x88CTP2vxJ8tna00Z6240B_YxgVKy

```
[13] import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler

data = {
    'Country': ['India', 'US', 'India', 'US', 'UK', 'India'],
    'Age': [22, 25, np.nan, 30, 28, 35],
    'Salary': [40000, 60000, 60000, np.nan, 72000, 50000],
    'Purchased': ['No', 'Yes', 'Yes', 'No', 'Yes', 'Yes']
}

df = pd.DataFrame(data)

# 1. Fill Missing Values
df['Age'] = df['Age'].fillna(df['Age'].mean())
df['Salary'] = df['Salary'].fillna(df['Salary'].mean())
print(df)
print("\n")

# 2. Encode Categorical Data
df_encoded = pd.get_dummies(df, columns=['Country'])
# Fix: Changed 'yes' to 'Yes' to match your data
df_encoded['Purchased'] = df_encoded['Purchased'].map({'Yes': 1, 'No': 0})
print(df_encoded)
print("\n")

# 3. Split Features and Target
X = df_encoded.drop('Purchased', axis=1)
Y = df_encoded['Purchased']
print("DATA AFTER ENCODING")
print(df_encoded)
print("\n")

# 4. Feature Scaling
Scaler = StandardScaler()
X_Scaled = Scaler.fit_transform(X)

print("Final processed data ready for AI:")
# Fix: Printing the actual scaled variable
print(pd.DataFrame(X_Scaled, columns=X.columns))
```

Hot weather
Now

colab.research.google.com/drive/1JYQvmyBSJD1k6KMSYhNzysU8ccC2ifg

Reshmitha 339

File Edit View Insert Runtime Tools Help

Q Commands + Code + Text ▶ Run all

```
[1] print(x_scaled_df)
```

ORIGINAL RAW DATA

	Country	Age	Salary	Purchased
0	India	22.0	40000.0	No
1	US	25.0	60000.0	Yes
2	India	NaN	60000.0	Yes
3	US	30.0	NaN	No
4	UK	28.0	72000.0	Yes
5	India	35.0	50000.0	Yes

MISSING VALUES COUNT

	Country	Age	Salary	Purchased
	0	1	1	0

dtype: int64

Country Age Salary Purchased

	Country	Age	Salary	Purchased
0	India	22.0	40000.0	No
1	US	25.0	60000.0	Yes
2	India	28.0	60000.0	Yes
3	US	30.0	50000.0	No
4	UK	28.0	72000.0	Yes
5	India	35.0	50000.0	Yes

DATA AFTER ENCODING

	Age	Salary	Purchased	Country_India	Country_UK	Country_US
0	22.0	40000.0	0	True	False	False
1	25.0	60000.0	1	False	False	True
2	28.0	60000.0	1	True	False	False
3	30.0	50000.0	0	False	False	True
4	28.0	72000.0	1	False	True	False
5	35.0	50000.0	1	True	False	False

FINAL PROCESSED DATA READY FOR AI

	Age	Salary	Country_India	Country_UK	Country_US
0	-1.486615	-1.918006	1.0	-0.447214	-0.707107
1	-0.742307	0.213201	-1.0	-0.447214	1.414214
2	0.000000	0.213201	1.0	-0.447214	-0.707107
3	0.494872	0.000000	-1.0	-0.447214	1.414214
4	0.000000	1.492405	-1.0	2.236068	-0.707107
5	1.732051	0.000000	1.0	-0.447214	-0.707107

3RD EXPERIMENT LINEAR REGRESSION

The screenshot shows a Jupyter Notebook interface with a code cell containing the following Python code:

```
import pandas as pd
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn import datasets

# Load dataset
diabetes = datasets.load_diabetes()
X = diabetes.data
Y = diabetes.target

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)

# Fit models
lr = LinearRegression().fit(X_train, Y_train)
ridge = Ridge(alpha=1.0).fit(X_train, Y_train)
lasso = Lasso(alpha=0.1).fit(X_train, Y_train)

# Predict and print MSE - Corrected: X_test (capitalized)
print(f"Linear Regression MSE: {mean_squared_error(Y_test, lr.predict(X_test)):.2f}")
print(f"Ridge Regression MSE: {mean_squared_error(Y_test, ridge.predict(X_test)):.2f}")
print(f"Lasso Regression MSE: {mean_squared_error(Y_test, lasso.predict(X_test)):.2f}")
```

The output of the code cell is displayed below the code:

```
*** Linear Regression MSE: 2900.19
Ridge Regression MSE: 3077.42
Lasso Regression MSE: 2798.19
```

The interface includes a top bar with the user name "Reshmitha 339" and a menu with "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". A search bar and "Run all" button are also present. The bottom status bar shows the temperature "36°C Sunny" and the language "ENG IN".

4TH EXPERIMENT NAIVE BAYES

The screenshot shows a Jupyter Notebook interface with a code cell containing the following Python code:

```
import pandas as pd
from scipy.stats import scoreatpercentile
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB # corrected spelling
from sklearn.metrics import accuracy_score

data = {
    'Study Hours': [5,2,8,1,6,3,7,4],
    'Attendance': [80,50,90,40,85,60,88,65], # corrected spelling
    'Result': ['Pass', 'Fail', 'Pass', 'Fail', 'Pass', 'Fail', 'Pass', 'Fail'] # corrected 'False'
}

df = pd.DataFrame(data)
df['Result'] = df['Result'].str.lower()

X = df[['Study Hours', 'Attendance']]
Y = df['Result']

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.25, random_state=42)

model = GaussianNB()
model.fit(X_train, Y_train) # missing step added

Y_pred = model.predict(X_test)

accuracy = accuracy_score(Y_test, Y_pred) # corrected order

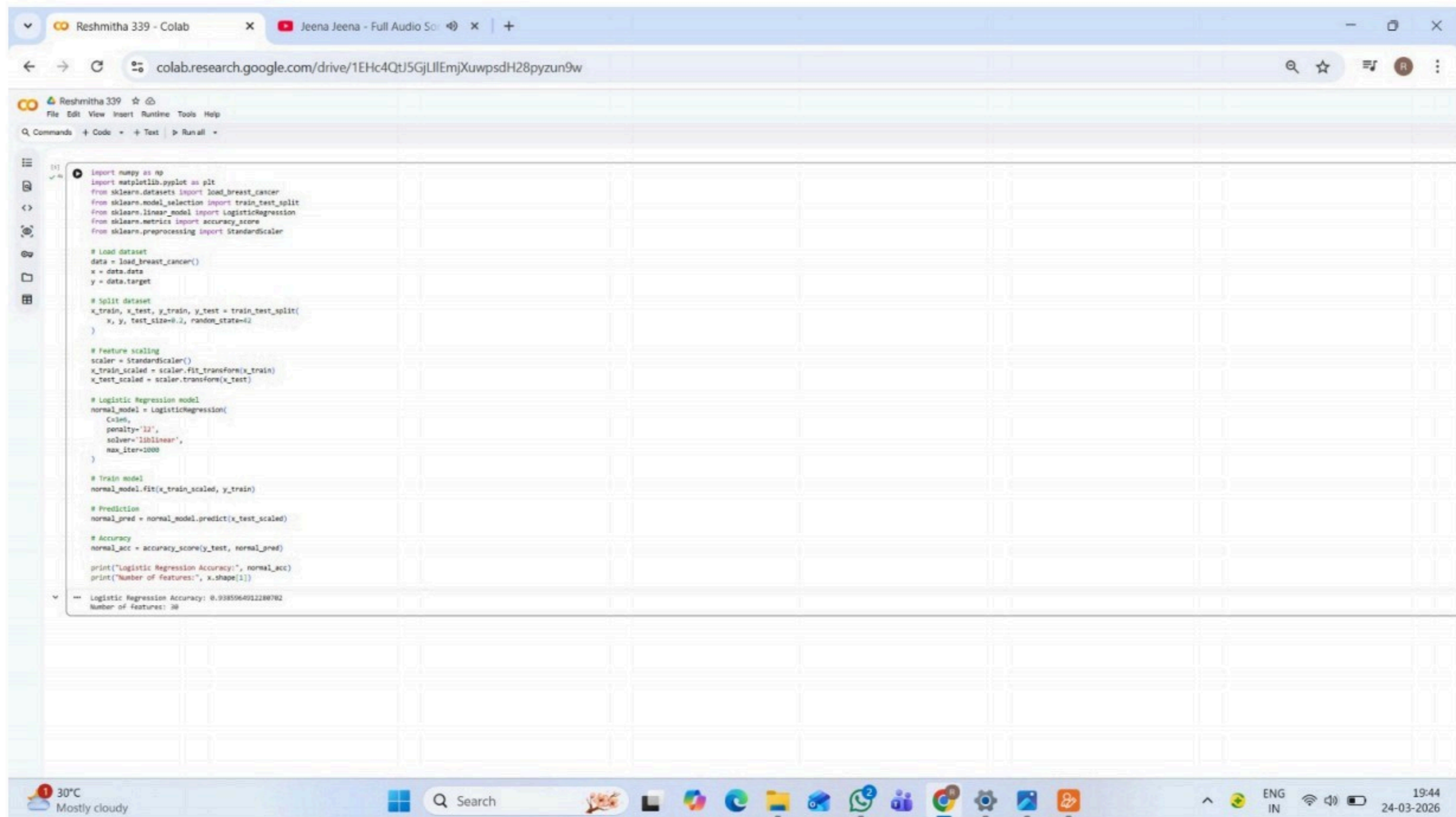
print("Predicted Results:", Y_pred)
print("Actual Results:", Y_test.values)
print("Accuracy:", accuracy)
```

The output of the code cell is displayed below the code:

```
*** Predicted Results: ['fail' 'fail']
Actual Results: ['fail' 'fail']
Accuracy: 1.0
```

The interface includes a top bar with the user name "Reshmitha 339" and a menu with "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". A search bar and "Run all" button are also present. The bottom status bar shows the temperature "36°C Sunny" and the language "ENG IN".

5TH IMPLEMENT REGULARIZED LOGISTIC REGULARIZATION



The screenshot shows a Google Colab notebook with the following code:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler

# Load dataset
data = load_breast_cancer()
x = data.data
y = data.target

# Split dataset
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42
)

# Feature scaling
scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)

# Logistic Regression model
normal_model = LogisticRegression(
    C=100,
    penalty='l2',
    solver='liblinear',
    max_iter=1000
)

# Train model
normal_model.fit(x_train_scaled, y_train)

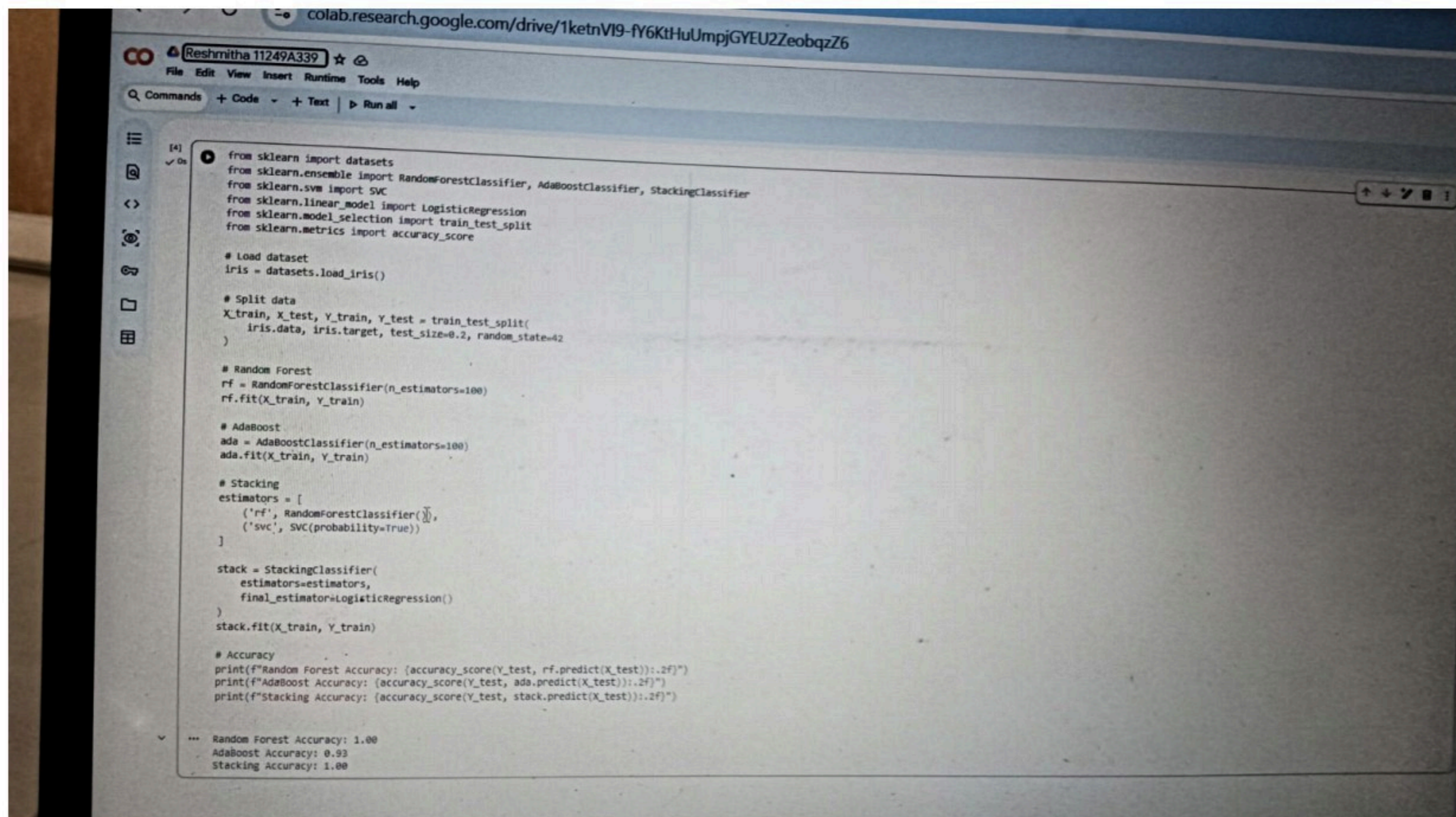
# Prediction
normal_pred = normal_model.predict(x_test_scaled)

# Accuracy
normal_acc = accuracy_score(y_test, normal_pred)
print("Logistic Regression Accuracy:", normal_acc)
print("Number of features:", x.shape[1])
```

The output of the code is:

```
Logistic Regression Accuracy: 0.989564012280702
Number of features: 30
```

6TH EXPERIMENT BUILDS MODELS USING DIFFERENT ENSEMBLING



The screenshot shows a Google Colab notebook with the following code:

```
from sklearn import datasets
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, StackingClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load dataset
iris = datasets.load_iris()

# Split data
X_train, X_test, Y_train, Y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42
)

# Random Forest
rf = RandomForestClassifier(n_estimators=100)
rf.fit(X_train, Y_train)

# AdaBoost
ada = AdaBoostClassifier(n_estimators=100)
ada.fit(X_train, Y_train)

# Stacking
estimators = [
    ('rf', RandomForestClassifier(n_estimators=100)),
    ('svc', SVC(probability=True))
]

stack = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression()
)
stack.fit(X_train, Y_train)

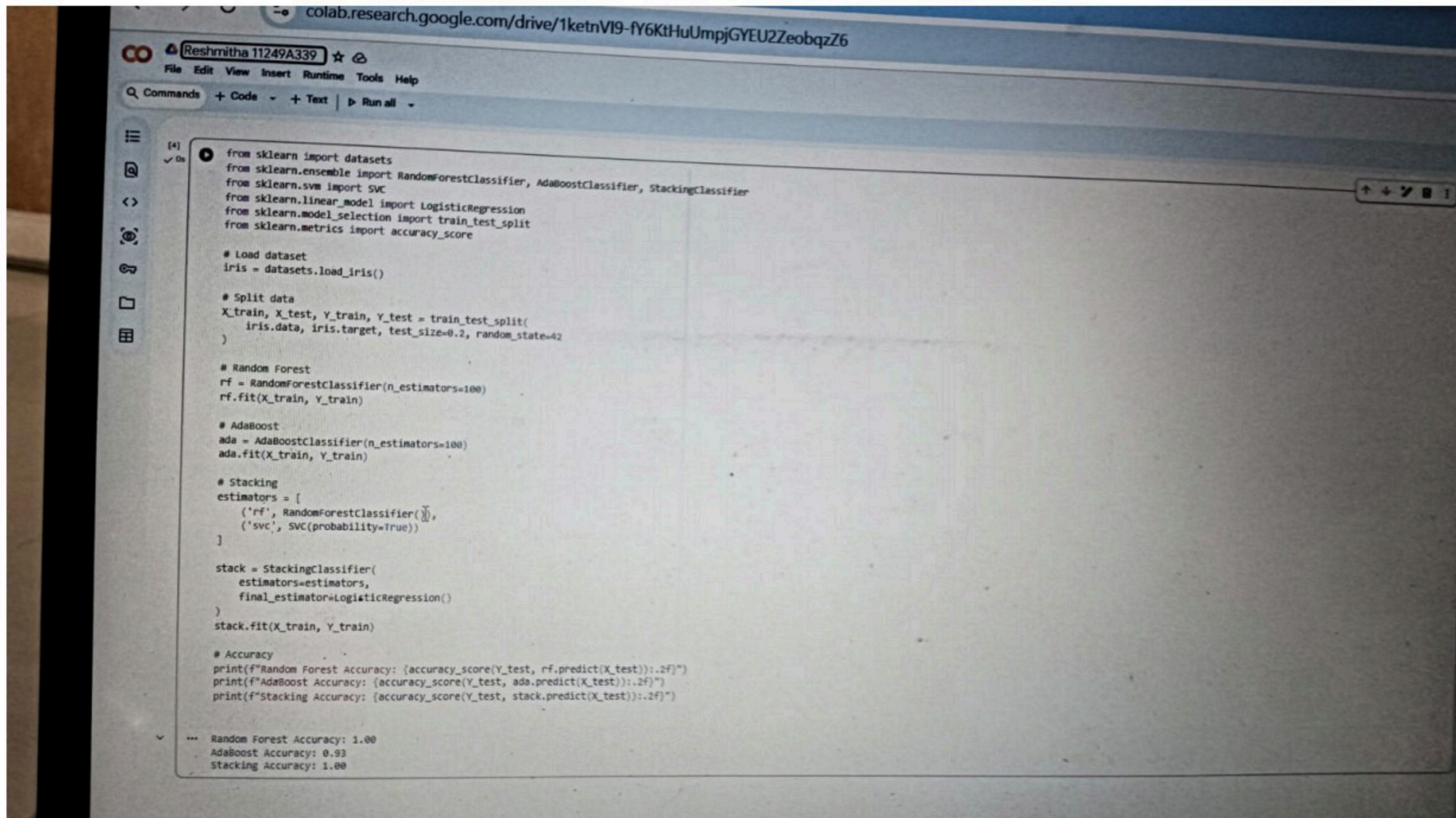
# Accuracy
print(f"Random Forest Accuracy: {accuracy_score(Y_test, rf.predict(X_test)):.2f}")
print(f"AdaBoost Accuracy: {accuracy_score(Y_test, ada.predict(X_test)):.2f}")
print(f"Stacking Accuracy: {accuracy_score(Y_test, stack.predict(X_test)):.2f}")
```

The output of the code is:

```
Random Forest Accuracy: 1.00
AdaBoost Accuracy: 0.93
Stacking Accuracy: 1.00
```

7TH EXPERIMENT IMPLEMENT DECISION TREE

ALGORITHM FOR CLASSIFICATION AND REGRESSION



```
from sklearn import datasets
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, StackingClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Load dataset
iris = datasets.load_iris()

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=42
)

# Random Forest
rf = RandomForestClassifier(n_estimators=100)
rf.fit(X_train, y_train)

# AdaBoost
ada = AdaBoostClassifier(n_estimators=100)
ada.fit(X_train, y_train)

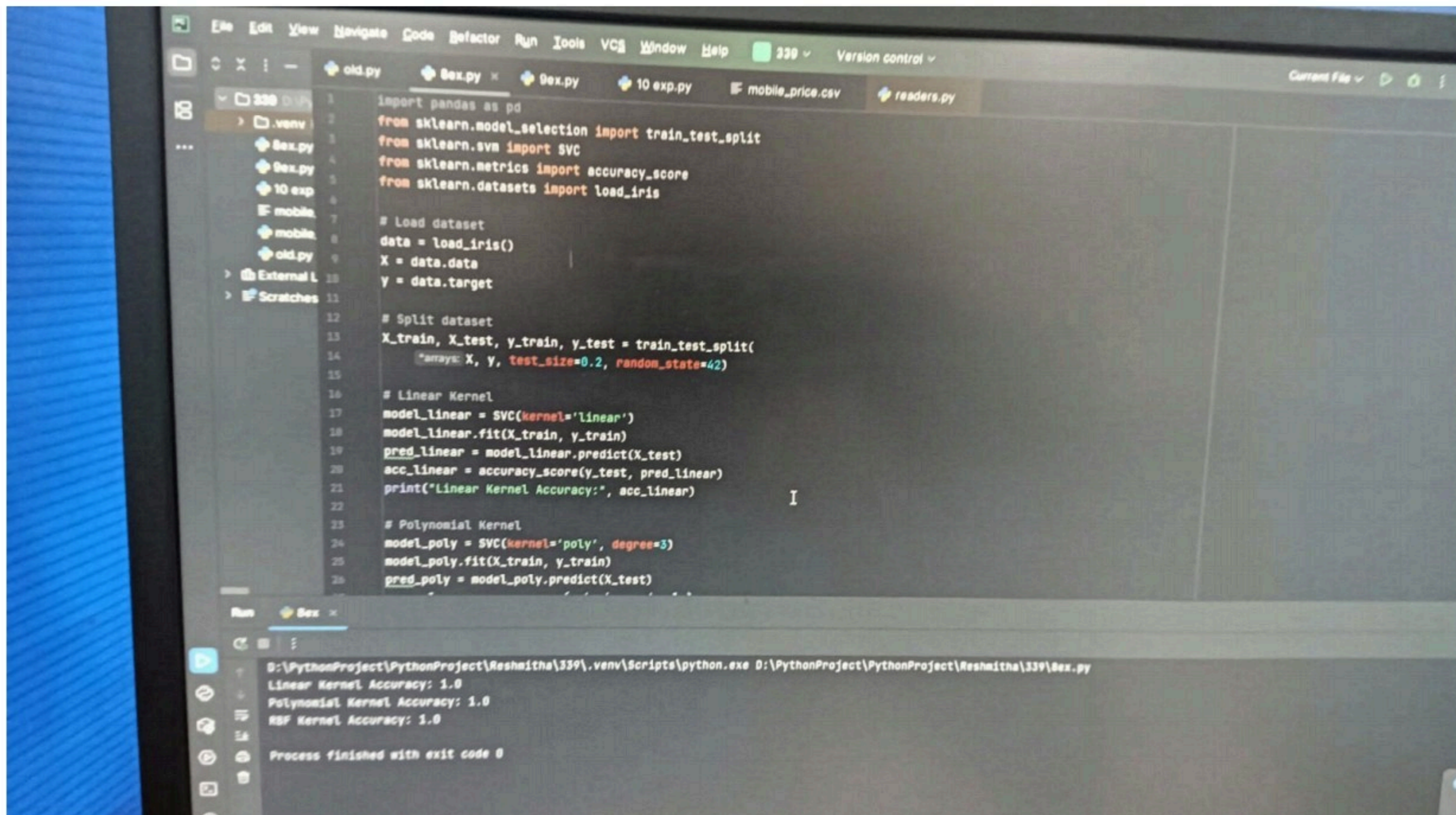
# Stacking
estimators = [
    ('rf', RandomForestClassifier(10)),
    ('svc', SVC(probability=True))
]

stack = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression()
)
stack.fit(X_train, y_train)

# Accuracy
print(f"Random Forest Accuracy: {accuracy_score(y_test, rf.predict(X_test)):.2f}")
print(f"AdaBoost Accuracy: {accuracy_score(y_test, ada.predict(X_test)):.2f}")
print(f"Stacking Accuracy: {accuracy_score(y_test, stack.predict(X_test)):.2f}")
```

Random Forest Accuracy: 1.00
AdaBoost Accuracy: 0.93
Stacking Accuracy: 1.00

8TH EXPERIMENT KERNEL FUNCTIONS



```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris

# Load dataset
data = load_iris()
X = data.data
y = data.target

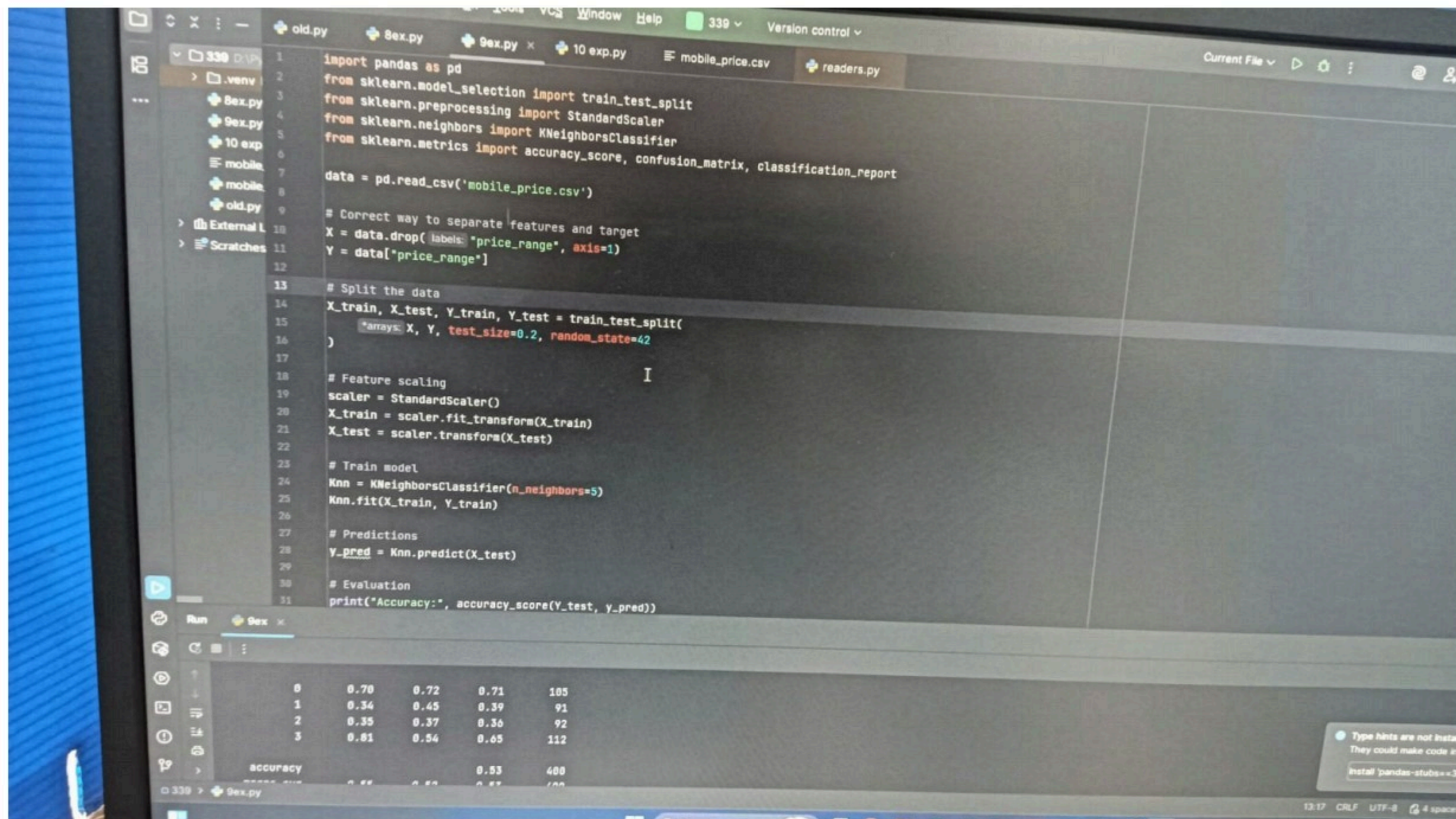
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Linear Kernel
model_linear = SVC(kernel='linear')
model_linear.fit(X_train, y_train)
pred_linear = model_linear.predict(X_test)
acc_linear = accuracy_score(y_test, pred_linear)
print("Linear Kernel Accuracy:", acc_linear)

# Polynomial Kernel
model_poly = SVC(kernel='poly', degree=3)
model_poly.fit(X_train, y_train)
pred_poly = model_poly.predict(X_test)
```

Linear Kernel Accuracy: 1.0
Polynomial Kernel Accuracy: 1.0
RBF Kernel Accuracy: 1.0
Process finished with exit code 0

9TH EXPERIMENT K NEAREST NEIGHBORS(KNN)



```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

data = pd.read_csv('mobile_price.csv')

# Correct way to separate features and target
X = data.drop(labels="price_range", axis=1)
Y = data["price_range"]

# Split the data
X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.2, random_state=42
)

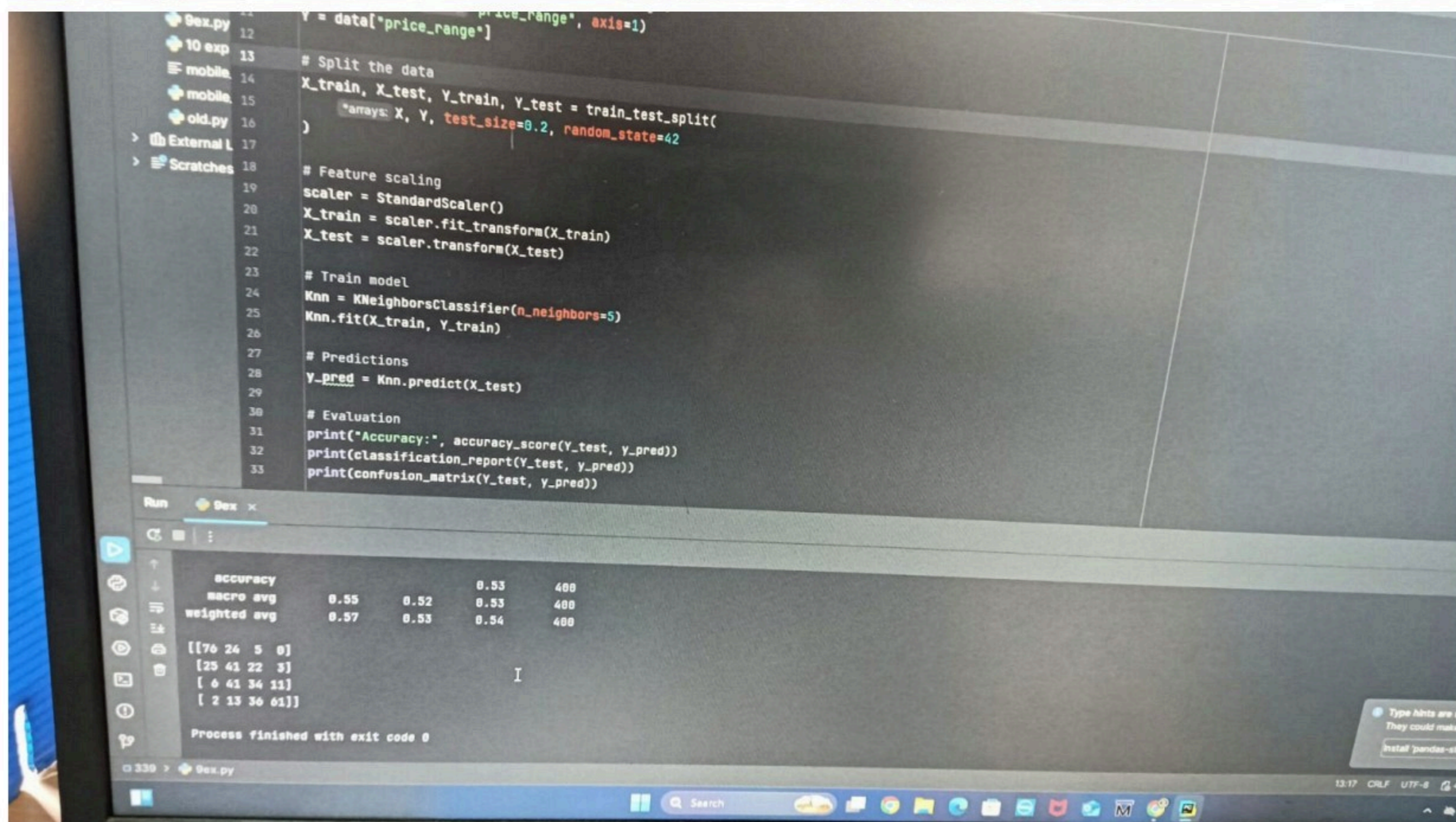
# Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train model
Knn = KNeighborsClassifier(n_neighbors=5)
Knn.fit(X_train, Y_train)

# Predictions
y_pred = Knn.predict(X_test)

# Evaluation
print("Accuracy:", accuracy_score(Y_test, y_pred))
```

	0	0.70	0.72	0.71	105
1	0.34	0.45	0.39	91	
2	0.35	0.37	0.36	92	
3	0.61	0.54	0.65	112	
accuracy			0.53	400	



```
Y = data["price_range"]
# Split the data
X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.2, random_state=42
)

# Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train model
Knn = KNeighborsClassifier(n_neighbors=5)
Knn.fit(X_train, Y_train)

# Predictions
y_pred = Knn.predict(X_test)

# Evaluation
print("Accuracy:", accuracy_score(Y_test, y_pred))
print(classification_report(Y_test, y_pred))
print(confusion_matrix(Y_test, y_pred))
```

	accuracy	macro avg	weighted avg	
	0.55	0.52	0.53	400
	0.57	0.53	0.54	400

```
[[76 24 5 0]
 [25 41 22 3]
 [ 6 41 34 11]
 [ 2 13 36 61]]
```

Process finished with exit code 0

MOBILE PRICE CSV

```
1 battery_power,blue,clock_speed,dual_sim,fc,four_g,int_memory,m_dep,mobile_wt,n_cores,pc,px_height,px_width,ram,sc_h,sc_w,ta
2 842,0.2,2,0,1,0,7,0.6,188,2,2,20,756,2549,9,7,19,0,0,1,1
3 1021,1,0.5,1,0,1,53,0.7,136,3,6,905,1988,2631,17,3,7,1,1,0,2
4 563,1,0.5,1,2,1,41,0.9,145,5,6,1263,1716,2603,11,2,9,1,1,0,2
5 615,1,2,5,0,0,0,10,0.8,131,6,9,1216,1786,2769,16,8,11,1,0,0,2
6 1821,1,1,2,0,13,1,44,0.6,141,2,14,1208,1212,1411,8,2,15,1,1,0,1
7 1859,0,0.5,1,3,0,22,0.7,164,1,7,1004,1654,1067,17,1,10,1,0,0,1
8 1821,0,1,7,0,4,1,10,0.8,139,8,10,381,1018,3220,13,8,18,1,0,1,3
9 1954,0,0.5,1,0,0,24,0.8,187,4,0,512,1149,700,16,3,5,1,1,1,0
10 1445,1,0.5,0,0,0,53,0.7,174,7,14,386,836,1099,17,1,20,1,0,0,0
11 509,1,0.6,1,2,1,9,0.1,93,5,15,1137,1224,513,19,10,12,1,0,0,0
12 769,1,2,9,1,0,0,9,0.1,182,5,1,248,874,3946,5,2,7,0,0,0,3
13 1520,1,2,2,0,5,1,33,0.5,177,8,18,151,1005,3826,14,9,13,1,1,1,3
14 1815,0,2,8,0,2,0,33,0.6,159,4,17,607,748,1482,18,0,2,1,0,0,1
15 803,1,2,1,0,7,0,17,1,0,198,4,11,344,1440,2680,7,1,4,1,0,1,2
16 1866,0,0.5,0,13,1,52,0.7,185,1,17,356,563,373,14,9,3,1,0,1,0
17 775,0,1,0,0,3,0,46,0.7,159,2,16,862,1864,568,17,15,11,1,1,1,0
18 838,0,0.5,0,1,1,13,0.1,196,8,4,984,1850,3554,10,9,19,1,0,1,3
19 595,0,0,9,1,7,1,23,0.1,121,3,17,441,810,3752,10,2,18,1,1,0,3
20 1131,1,0.5,1,11,0,49,0.6,101,5,18,658,878,1835,19,13,16,1,1,0,1
21 682,1,0.5,0,4,0,19,1,0,121,4,11,902,1064,2337,11,1,18,0,1,1,1
22 772,0,1,1,1,12,0,39,0.8,81,7,14,1314,1854,2819,17,15,3,1,1,0,3
23 1709,1,2,1,0,1,0,13,1,0,156,2,2,974,1385,3283,17,1,15,1,0,0,3
24 1949,0,2,6,1,4,0,47,0.3,199,4,7,407,822,1433,11,5,20,0,0,1,1
25 1602,1,2,8,1,4,1,38,0.7,114,3,20,466,788,1037,8,7,20,1,0,0,0
```

accuracy	macro avg
0.55	0.52
0.53	0.51
488	488

10TH EXPERIMENT Principal Component Analysis (PCA)
10TH EXPERIMENT Principal Component Analysis (PCA)

```
Project Files
D:\PythonProject\PythonProject\Reshma\
.venv
8sex.py
9sex.py
10exp.py
customers_half.csv.py
mobile_price.csv
mobile_price.csv.py
old.py
Extensions
Scratches

old.py
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.cluster import KMeans
5 data = pd.read_csv("Mall_Customers.csv")
6 X = data[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']]
7 scaler = StandardScaler()
8 X_scaled = scaler.fit_transform(X)
9 pca = PCA(n_components=2)
10 X_pca = pca.fit_transform(X_scaled)
11 print(pca.explained_variance_ratio_)
12 wcss = []
13
14 for i in range(1, 11):
15     km = KMeans(n_clusters=i)
16     km.fit(X_pca)
17     wcss.append(km.inertia_)
18 plt.plot(range(1, 11), wcss, marker='o')
19 plt.xlabel("Number of clusters")
20 plt.ylabel("WCSS")
21 plt.show()
22 kmeans = KMeans(n_clusters=5, random_state=0)
23 clusters = kmeans.fit_predict(X_pca)
24 plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters)
25 plt.xlabel("PCA Component 1")
26 plt.ylabel("PCA Component 2")
27 plt.show()
28
29
```

Run 10exp

D:\PythonProject\PythonProject\Reshma\339\.venv\Scripts\python.exe D:\PythonProject\PythonProject\Reshma\339\10exp.py [9.44266167 0.33306378]

